

Decision stability under reseeding, at realistic misfit

How often does the same data give a different answer?

Magnus Johansson

2026-08-06

Table of contents

Scope	1
Prediction being tested	1
Design	1
Arms (identical to the poly_weak study)	2
Run	3
Instability of the HDCl rule	287
Instability of the FWER and BH rules	288
Probability that two analysts disagree	289
Spread of the adjusted p-value	290
Spread of the HDCl bounds	290

Scope

Error rates averaged over datasets can look fine while individual decisions remain unstable. This study holds the **dataset fixed** and varies only the bootstrap seed, isolating Monte-Carlo noise from sampling noise: if an analyst re-runs `RMitemInfitCutoff()` on the same data with a different seed, how often does the flag change?

An earlier version of this exists in `infit_fwer_iterations.qmd` (Study B), but it has two limitations that this run addresses:

1. It used **dichotomous** items at $\rho = 0.15$ — misfit roughly 60 percent stronger than anything observed in real phq9 data, which we now know saturates detection.
2. It covered only n in $\{250, 500\}$ and $B \leq 1000$.

This matters because the current recommendation of $B \geq 1000$ for FWER rests entirely on that single **measurement** — stability, not calibration, is what drives it. Calibration is already nominal from $B \approx 400$.

Prediction being tested

Instability should be **worse** at realistic misfit, because when the signal is weaker more items sit near the decision boundary, and a boundary-adjacent item is exactly the one a different seed can push either way. Three levels are included so this is a dose-response test rather than a two-point comparison: $\rho = 0.15$ (the unrealistically strong condition used in the earlier study), $\rho = 0.50$ (intermediate), and $\rho = 0.85$ (calibrated to real phq9 misfit, injected `infit` ~ 1.25 against $q3 = 1.23$ and $q8 = 1.26$).

Design

- Same two arms as `infit_width_iterations_poly_weak.qmd`: **sim15** (15 stylised polytomous items) and **phq9** (9 items, CML parameters from the packaged real responses)
- ρ in $\{0.15, 0.50, 0.85\}$ for the misfit conditions; $k = 0$ has no ρ
- Misfitting items: the maximum per arm (3 for sim15, 2 for phq9), i.e. the strongest cascade, plus a $k = 0$ null baseline
- n in $\{250, 1000\}$
- **10 fixed datasets** per condition, each analysed with **25 bootstrap seeds** (seeds resolve instability better than datasets do — an item flagged with probability 0.1 is caught as non-unanimous 93 percent of the time with 25 seeds, only 79 percent with 15)
- B in $\{100, 400, 1000, 2000\}$; widths $\{.90, .95, .99, .995, .999\}$ read off the same draws
- FWER and BH p-values recorded alongside

Outcomes: the share of items whose flag is not unanimous across seeds, the probability that two randomly chosen seeds disagree, and the spread of the adjusted p-value and interval bounds.

```
suppressMessages({
  source("../R/helpers.R") # package internals and shared helper functions
  library(easyRasch2)      # RMitemInfit(), RMitemInfitCutoff()
  library(ggdist)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
NS      <- c(250, 1000)
BS      <- c(100, 400, 1000, 2000)
RHOS    <- c(0.15, 0.50, 0.85)
N_DATA  <- 10 # 3 -> 6 on 2026-07-31, 6 -> 10 on 2026-08-04 to tighten
          # the two-analyst disagreement contrasts; datasets 7-10 resume
N_SEEDS <- 25 # bootstrap seeds per dataset
NCORES  <- 10
ALPHA   <- 0.05
CHUNK   <- 30 # frequent checkpoints: this run may be stopped mid-way
RES      <- "infit_decision_stability.rds"
```

```
# run_resumable() is defined in ../R/helpers.R
```

Arms (identical to the poly_weak study)

```
S15_LOCS <- seq(-2, 2, length.out = 15)
S15_THR  <- lapply(S15_LOCS, function(L) L + seq(-1, 1, length.out = 3))
S15_MIS  <- vapply(c(0, -1, -2), function(tg) which.min(abs(S15_LOCS - tg)),
                  integer(1))

ph <- load_phq9_arm()
PH_THR <- ph$thr
PH_LOCS <- ph$locs
PH_SD   <- ph$sd
PH_MIS  <- c(5L, 8L)

ARMS <- list(
  sim15 = list(thr = S15_THR, sd = 1.5, mis = S15_MIS, locs = S15_LOCS,
              nm = paste0("V", 1:15), kmax = 3L),
  phq9  = list(thr = PH_THR, sd = PH_SD, mis = PH_MIS, locs = PH_LOCS,
              nm = paste0("q", 1:9), kmax = 2L)
)

sim_arm <- function(arm, n, k, rho, seed) {
  a <- ARMS[[arm]]
  set.seed(seed)
  S <- matrix(c(a$sd^2, rho * a$sd^2, rho * a$sd^2, a$sd^2), 2, 2)
  th <- MASS::mvrnorm(n, mu = c(0, 0), Sigma = S)
  ni <- length(a$thr); mis <- a$mis[seq_len(k)]
  d <- matrix(0L, n, ni); fit_items <- setdiff(seq_len(ni), mis)
  if (length(fit_items)) d[, fit_items] <- sim_partial_score(a$thr[fit_items], th[, 1])
}
```

```

if (length(mis))      d[, mis]      <- sim_partial_score(a$thr[mis], th[, 2])
d <- as.data.frame(d); names(d) <- a$nm; d
}

```

bounds_at() and flag_code() are defined in ../R/helpers.R

Run

Each cell is one (dataset, seed) pair: the dataset is regenerated from a seed that depends only on (arm, k, n, rho, dataset), so it is **identical across all bootstrap seeds**, while the bootstrap seed varies.

```

one_cell <- function(g) {
  out <- try({
    arm <- as.character(g$arm); k <- g$k; n <- g$n; rho <- g$rho
    dat <- g$dataset; sd_seed <- g$seed
    d <- sim_arm(arm, n, k, rho,
      seed = 5e8 + 1e7 * match(arm, names(ARMS)) + 1e6 * k +
      1e4 * round(rho * 100) + 1e3 * n + dat)
    items <- names(d)
    obs <- RMitemInfit(d, output = "dataframe")
    msq <- obs$Infit_MSQ[match(items, obs$Item)]
    hd <- list(); pv <- list()
    for (j in seq_along(BS)) {
      co <- try(RMitemInfitCutoff(d, iterations = BS[j], parallel = FALSE,
        seed = sd_seed * 1000 + j, verbose = FALSE),
        silent = TRUE)
      if (inherits(co, "try-error")) next
      hd[[length(hd) + 1L]] <- do.call(rbind, lapply(WIDTHS, function(w) {
        b <- bounds_at(co$results, w, items)
        data.frame(B = BS[j], w = w, item = seq_along(items),
          code = flag_code(msq, b[1, ], b[2, ]),
          lo = b[1, ], hi = b[2, ])
      })))
      fw <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
          alpha = ALPHA, output = "dataframe")), silent = TRUE)
      bh <- try(suppressWarnings(
        RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
          alpha = ALPHA, output = "dataframe")), silent = TRUE)
      if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
        i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
        pv[[length(pv) + 1L]] <- data.frame(
          B = BS[j], item = seq_along(items),
          padj_fwer = fw$padj_infit[i1], padj_bh = bh$padj_infit[i2])
      }
    }
    list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

blocks <- list()
for (a in names(ARMS)) {
  # null baseline: rho is inert with no misfitting items
  blocks[[length(blocks) + 1L]] <- expand.grid(
    arm = a, k = 0L, rho = 0, n = NS, dataset = 1:N_DATA, seed = 1:N_SEEDS,
    stringsAsFactors = FALSE)
}

```

```

blocks[[length(blocks) + 1L]] <- expand.grid(
  arm = a, k = ARMS[[a]]$kmax, rho = RHOS, n = NS,
  dataset = 1:N_DATA, seed = 1:N_SEEDS, stringsAsFactors = FALSE)
}
grid <- do.call(rbind, blocks)[, c("arm", "k", "rho", "n", "dataset", "seed")]

# Explicit priority order, so a run stopped early still answers the main
# question. Blocks cost roughly the same (~2.3 h each on 10 cores):
# 1. rho = 0.85 -- the calibrated, realistic condition (the point of the run)
# 2. k = 0      -- instability under a true model, the reference baseline
# 3. rho = 0.15 -- the old unrealistically strong condition, for contrast
# 4. rho = 0.50 -- intermediate; fills in the dose-response, least critical
prio <- with(grid, ifelse(k == 0L, 2L,
  ifelse(rho == 0.85, 1L,
    ifelse(rho == 0.15, 3L, 4L))))
grid <- grid[order(prio, grid$n), ]
rownames(grid) <- NULL
cat("cells to run:", nrow(grid), "\n")

```

```
cells to run: 4000
```

```
print(table(priority = sort(prio)))
```

```

priority
  1    2    3    4
1000 1000 1000 1000

```

```

s <- run_resumable(RES, grid, c("arm", "k", "rho", "n", "dataset", "seed"),
  one_cell,
  extra = list(arms = lapply(ARMS, function(a)
    a[c("mis", "locs", "sd", "nm", "kmax"))]))

```

```
running 850 new cells (3150 cached)
```

```
checkpoint 3180 / 4000 (1389.2 min)
```

```
checkpoint 3210 / 4000 (1399.6 min)
```

```
checkpoint 3240 / 4000 (1408.8 min)
```

```
checkpoint 3270 / 4000 (1418.3 min)
```

```
checkpoint 3300 / 4000 (1427.5 min)
```

```
checkpoint 3330 / 4000 (1431.8 min)
```

```
checkpoint 3360 / 4000 (1436 min)
```

checkpoint 3390 / 4000 (1440.3 min)

checkpoint 3420 / 4000 (1457.3 min)

checkpoint 3450 / 4000 (1480.6 min)

checkpoint 3480 / 4000 (1504.2 min)

checkpoint 3510 / 4000 (1523.6 min)

checkpoint 3540 / 4000 (1534.5 min)

checkpoint 3570 / 4000 (1545.5 min)

checkpoint 3600 / 4000 (1556.5 min)

checkpoint 3630 / 4000 (1565.7 min)

checkpoint 3660 / 4000 (1574.8 min)

checkpoint 3690 / 4000 (1584 min)

checkpoint 3720 / 4000 (1589.9 min)

checkpoint 3750 / 4000 (1594.2 min)

checkpoint 3780 / 4000 (1598.4 min)

checkpoint 3810 / 4000 (1609.1 min)

checkpoint 3840 / 4000 (1632.7 min)

checkpoint 3870 / 4000 (1656.6 min)

checkpoint 3900 / 4000 (1680.3 min)

checkpoint 3930 / 4000 (1691.4 min)

checkpoint 3960 / 4000 (1702.6 min)

```
checkpoint 3990 / 4000 (1713.6 min)
```

```
checkpoint 4000 / 4000 (1717.3 min)
```

[illegible][illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
Warning in data.frame(..., check.names = FALSE): row names were found from a  
short variable and have been discarded
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
Warning in data.frame(..., check.names = FALSE): row names were found from a  
short variable and have been discarded
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
P <- bind_part("pval") |> mutate(rho = sprintf("%.2f", rho))
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
Warning in data.frame(..., check.names = FALSE): row names were found from a
short variable and have been discarded
```

```
cat(nrow(s$grid), "cells |", round(s$elapsed_min, 1), "min |",
    "failed:", sum(sapply(s$res, is.null)), "\n")
```

```
4000 cells | 1717.3 min | failed: 0
```

Instability of the HDCl rule

Share of items whose flag is **not unanimous** across the 25 seeds, averaged over the 5 datasets.

```
H |>
  group_by(arm, rho, k, n, w, B, dataset, item) |>
  summarise(rate = mean(flag), .groups = "drop") |>
  group_by(arm, rho, k, n, w, B) |>
  summarise(unstable = round(100 * mean(rate > 0 & rate < 1), 1), .groups = "drop") |>
  filter(w == "0.999") |>
  pivot_wider(names_from = B, values_from = unstable, names_prefix = "B=") |>
  arrange(arm, rho, n)
```

arm	rho	k	n	w	B=100	B=400	B=1000	B=2000
phq9	0.00	0	250	0.999	5.6	2.2	2.2	2.2
phq9	0.00	0	1000	0.999	5.6	0.0	0.0	0.0
phq9	0.15	2	250	0.999	37.8	28.9	20.0	20.0
phq9	0.15	2	1000	0.999	1.1	1.1	1.1	2.2
phq9	0.50	2	250	0.999	42.2	30.0	20.0	14.4
phq9	0.50	2	1000	0.999	40.0	37.8	27.8	28.9
phq9	0.85	2	250	0.999	13.3	7.8	5.6	4.4
phq9	0.85	2	1000	0.999	26.7	18.9	12.2	12.2
sim15	0.00	0	250	0.999	4.7	2.0	0.7	0.7
sim15	0.00	0	1000	0.999	5.3	1.3	0.0	0.7
sim15	0.15	3	250	0.999	48.0	36.0	25.3	22.7
sim15	0.15	3	1000	0.999	6.0	8.7	8.0	6.7
sim15	0.50	3	250	0.999	32.7	20.7	12.7	10.7
sim15	0.50	3	1000	0.999	44.7	46.0	31.3	28.0
sim15	0.85	3	250	0.999	18.7	6.0	4.0	2.7
sim15	0.85	3	1000	0.999	22.0	12.0	7.3	6.7

Instability of the FWER and BH rules

```
P |>
  mutate(fwer = as.integer(padj_fwer < ALPHA),
         bh = as.integer(padj_bh < ALPHA)) |>
  pivot_longer(c(fwer, bh), names_to = "rule", values_to = "flag") |>
  group_by(arm, rho, k, n, rule, B, dataset, item) |>
  summarise(rate = mean(flag, na.rm = TRUE), .groups = "drop") |>
  group_by(arm, rho, k, n, rule, B) |>
  summarise(unstable = round(100 * mean(rate > 0 & rate < 1), 1), .groups = "drop") |>
  pivot_wider(names_from = B, values_from = unstable, names_prefix = "B=") |>
  arrange(rule, arm, rho, n)
```

arm	rho	k	n	rule	B=100	B=400	B=1000	B=2000
phq9	0.00	0	250	bh	0.0	2.2	0.0	0.0
phq9	0.00	0	1000	bh	0.0	0.0	0.0	0.0
phq9	0.15	2	250	bh	26.7	12.2	10.0	8.9
phq9	0.15	2	1000	bh	0.0	0.0	0.0	0.0
phq9	0.50	2	250	bh	38.9	21.1	15.6	12.2
phq9	0.50	2	1000	bh	10.0	5.6	4.4	3.3
phq9	0.85	2	250	bh	6.7	3.3	3.3	1.1
phq9	0.85	2	1000	bh	25.6	11.1	4.4	4.4
sim15	0.00	0	250	bh	0.0	0.7	0.7	0.7
sim15	0.00	0	1000	bh	0.0	0.0	0.0	0.0
sim15	0.15	3	250	bh	38.0	17.3	8.7	6.0
sim15	0.15	3	1000	bh	0.0	0.0	0.0	0.0
sim15	0.50	3	250	bh	33.3	14.0	11.3	8.0
sim15	0.50	3	1000	bh	22.7	9.3	8.0	3.3
sim15	0.85	3	250	bh	4.0	6.0	3.3	2.0
sim15	0.85	3	1000	bh	22.0	8.7	6.0	4.0
phq9	0.00	0	250	fwer	2.2	0.0	0.0	0.0
phq9	0.00	0	1000	fwer	1.1	0.0	0.0	0.0
phq9	0.15	2	250	fwer	30.0	20.0	16.7	12.2
phq9	0.15	2	1000	fwer	0.0	0.0	0.0	0.0
phq9	0.50	2	250	fwer	23.3	12.2	11.1	8.9
phq9	0.50	2	1000	fwer	16.7	12.2	11.1	6.7
phq9	0.85	2	250	fwer	6.7	3.3	2.2	1.1
phq9	0.85	2	1000	fwer	12.2	6.7	3.3	2.2
sim15	0.00	0	250	fwer	1.3	0.7	0.7	0.0
sim15	0.00	0	1000	fwer	0.7	0.0	0.0	0.0
sim15	0.15	3	250	fwer	40.0	16.0	8.7	8.0

arm	rho	k	n	rule	B=100	B=400	B=1000	B=2000
sim15	0.15	3	1000	fwer	0.0	0.0	0.0	0.0
sim15	0.50	3	250	fwer	12.7	3.3	2.0	1.3
sim15	0.50	3	1000	fwer	32.0	17.3	9.3	6.7
sim15	0.85	3	250	fwer	4.7	1.3	1.3	0.7
sim15	0.85	3	1000	fwer	7.3	3.3	2.0	0.7

Probability that two analysts disagree

For an item flagged with probability p across seeds, two independently seeded runs disagree with probability $2p(1-p)$. Averaged over items, this is the chance that two analysts running the same data reach a different conclusion about at least one item.

```
P |>
  mutate(fwer = as.integer(padj_fwer < ALPHA)) |>
  group_by(arm, rho, k, n, B, dataset, item) |>
  summarise(p = mean(fwer, na.rm = TRUE), .groups = "drop") |>
  mutate(disagree = 2 * p * (1 - p)) |>
  group_by(arm, rho, k, n, B) |>
  summarise(any_item = round(100 * (1 - prod(1 - disagree)^(1 / n_distinct(dataset))), 1),
            per_item = round(100 * mean(disagree), 2), .groups = "drop") |>
  pivot_wider(names_from = B, values_from = c(per_item, any_item)) |>
  arrange(arm, rho, n)
```

arm	rho	k	per_item_100	per_item_400	per_item_1000	per_item_2000	any_item_100	any_item_400	any_item_1000	any_item_2000
phq9	0.00	250	0.57	0.00	0.00	0.00	5.9	0.0	0.0	0.0
phq9	0.00	1000	0.09	0.00	0.00	0.00	0.8	0.0	0.0	0.0
phq9	0.15	250	9.63	6.43	4.81	3.19	67.0	53.2	42.3	29.7
phq9	0.15	1000	0.00	0.00	0.00	0.00	0.0	0.0	0.0	0.0
phq9	0.50	250	7.35	4.39	3.09	1.69	57.0	39.7	29.2	16.6
phq9	0.50	1000	6.09	3.88	2.85	2.02	51.5	35.4	27.3	20.2
phq9	0.85	250	1.58	0.47	0.33	0.16	15.8	4.6	3.1	1.6
phq9	0.85	1000	3.57	1.96	0.78	0.60	33.2	19.6	8.5	6.7
sim15	0.00	250	0.26	0.14	0.05	0.00	4.5	2.3	0.8	0.0
sim15	0.00	1000	0.14	0.00	0.00	0.00	2.3	0.0	0.0	0.0
sim15	0.15	250	10.69	4.76	2.49	1.83	86.6	59.4	37.2	27.6
sim15	0.15	1000	0.00	0.00	0.00	0.00	0.0	0.0	0.0	0.0
sim15	0.50	250	2.89	0.69	0.43	0.31	41.1	11.5	8.0	5.3
sim15	0.50	1000	9.22	4.31	2.87	1.83	82.9	55.1	42.6	29.1
sim15	0.85	250	1.00	0.46	0.41	0.24	16.6	8.5	7.5	4.4
sim15	0.85	1000	2.10	0.72	0.52	0.33	32.7	12.4	9.6	6.7

Spread of the adjusted p-value

```
P |>
  group_by(arm, rho, k, n, B, dataset, item) |>
  summarise(sd_p = sd(padj_fwer, na.rm = TRUE), .groups = "drop") |>
  group_by(arm, rho, k, n, B) |>
  summarise(median_sd = round(median(sd_p, na.rm = TRUE), 4), .groups = "drop") |>
  pivot_wider(names_from = B, values_from = median_sd, names_prefix = "B=") |>
  arrange(arm, rho, n)
```

arm	rho	k	n	B=100	B=400	B=1000	B=2000
phq9	0.00	0	250	0.0182	0.0083	0.0055	0.0036
phq9	0.00	0	1000	0.0164	0.0078	0.0049	0.0038
phq9	0.15	2	250	0.0183	0.0096	0.0067	0.0044
phq9	0.15	2	1000	0.0000	0.0000	0.0000	0.0000
phq9	0.50	2	250	0.0461	0.0242	0.0171	0.0105
phq9	0.50	2	1000	0.0027	0.0017	0.0009	0.0006
phq9	0.85	2	250	0.0383	0.0222	0.0162	0.0104
phq9	0.85	2	1000	0.0662	0.0315	0.0229	0.0164
sim15	0.00	0	250	0.0065	0.0046	0.0031	0.0021
sim15	0.00	0	1000	0.0055	0.0028	0.0018	0.0014
sim15	0.15	3	250	0.0328	0.0191	0.0115	0.0091
sim15	0.15	3	1000	0.0000	0.0000	0.0000	0.0000
sim15	0.50	3	250	0.0541	0.0314	0.0210	0.0149
sim15	0.50	3	1000	0.0102	0.0051	0.0034	0.0024
sim15	0.85	3	250	0.0162	0.0095	0.0056	0.0040
sim15	0.85	3	1000	0.0250	0.0118	0.0100	0.0059

Spread of the HDCI bounds

```
H |>
  filter(w == "0.999") |>
  group_by(arm, rho, k, n, B, dataset, item) |>
  summarise(sd_hi = sd(hi), .groups = "drop") |>
  group_by(arm, rho, k, n, B) |>
  summarise(median_sd_upper = round(median(sd_hi), 4), .groups = "drop") |>
  pivot_wider(names_from = B, values_from = median_sd_upper, names_prefix = "B=") |>
  arrange(arm, rho, n)
```

arm	rho	k	n	B=100	B=400	B=1000	B=2000
phq9	0.00	0	250	0.0411	0.0370	0.0275	0.0231
phq9	0.00	0	1000	0.0178	0.0157	0.0119	0.0105
phq9	0.15	2	250	0.0360	0.0327	0.0229	0.0204
phq9	0.15	2	1000	0.0163	0.0152	0.0108	0.0097

arm	rho	k	n	B=100	B=400	B=1000	B=2000
phq9	0.50	2	250	0.0394	0.0335	0.0250	0.0221
phq9	0.50	2	1000	0.0176	0.0154	0.0109	0.0104
phq9	0.85	2	250	0.0406	0.0339	0.0259	0.0232
phq9	0.85	2	1000	0.0185	0.0168	0.0120	0.0108
sim15	0.00	0	250	0.0442	0.0386	0.0283	0.0239
sim15	0.00	0	1000	0.0200	0.0180	0.0129	0.0116
sim15	0.15	3	250	0.0375	0.0333	0.0245	0.0222
sim15	0.15	3	1000	0.0172	0.0157	0.0118	0.0101
sim15	0.50	3	250	0.0409	0.0353	0.0261	0.0231
sim15	0.50	3	1000	0.0182	0.0164	0.0123	0.0107
sim15	0.85	3	250	0.0422	0.0377	0.0276	0.0246
sim15	0.85	3	1000	0.0197	0.0171	0.0124	0.0113

```
P |>
  mutate(fwer = as.integer(padj_fwer < ALPHA)) |>
  group_by(arm, rho, k, n, B, dataset, item) |>
  summarise(rate = mean(fwer, na.rm = TRUE), .groups = "drop") |>
  group_by(arm, rho, n, B) |>
  summarise(unstable = 100 * mean(rate > 0 & rate < 1), .groups = "drop") |>
  ggplot(aes(factor(B), unstable, colour = rho, group = rho)) +
  geom_line() + geom_point() +
  facet_grid(arm ~ n, labeller = label_both) +
  labs(x = "Bootstrap iterations (B)",
       y = "% of items with a non-unanimous FWER flag",
       colour = "rho") +
  theme_minimal(base_size = 10)
```

